

AI Specialization 3: Empirical Risk, MLE, EM, and Neural Networks

AI Specialization 3: SSU

Minimizing empirical risk. Maximum likelihood estimation, EM algorithm. Classical and deep neural networks and their learning.

Sources used: official CTU CourseWare page for BE4M33SSU lectures, which lists empirical risk, empirical risk minimization, SVMs, supervised learning for deep networks, SGD/deep convolutional networks, and generative learning/EM; CTU state-exam topic list; standard statistical learning material.

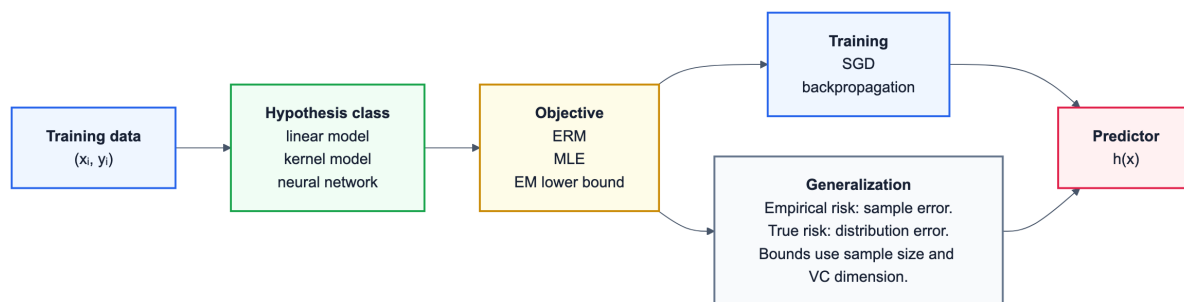


Figure 1: Supervised-learning pipeline: data and a hypothesis class define an objective, training optimizes it, and generalization controls performance on unseen data.

SSU in Plain English

SSU is about learning a predictor from data. The big story is: define loss, minimize training loss, control generalization, and understand the main model families.

1. Define the objective.

Risk, empirical risk, likelihood, or loss.

2. Pick the model.

SVM, probabilistic model, EM, neural network.

3. Explain training.

Optimization plus generalization.

ERM and generalization

Training error is not the same as true error.

- Risk = expected loss on the real distribution.
- Empirical risk = average loss on the sample.
- Bounds explain when low training loss transfers.

VC dimension

A capacity measure for hypothesis classes.

- High capacity can fit more patterns.
- Too much capacity risks overfitting.
- Consistency means convergence to optimal risk.

SVMs

Find a separating hyperplane with maximum margin.

- Hard margin: separable data.
- Soft margin: allows violations with penalty.
- Kernels make nonlinear boundaries via inner products.

MLE and EM

Fit probabilistic models by maximizing likelihood.

- MLE chooses parameters that make data likely.
- EM handles hidden variables.
- E-step estimates hidden assignments; M-step updates parameters.

Deep networks

Layered differentiable functions trained by gradients.

- Backprop computes gradients through layers.
- SGD updates parameters from mini-batches.
- CNNs use convolution for spatial structure.

How to answer

Tie every method to loss and optimization.

- What is minimized or maximized?
- What are the parameters?
- How do we avoid overfitting?

Exam sentence: ERM minimizes training loss, but learning theory asks when this also gives low true risk; VC dimension and concentration bounds control that gap.

Three traps

- Empirical risk is measured on the sample; true risk is an expectation over the unknown distribution.
- EM increases a likelihood lower bound, not by magic "guessing".
- Backprop computes gradients; SGD is the optimizer using them.

Use this page to connect formulas to the learning story.

1. Empirical Risk Minimization

Prediction setup

Let examples (x, y) be drawn from unknown distribution $P(X, Y)$. A predictor $h : X \rightarrow Y$ is chosen from hypothesis class H . A loss function $\ell(h(x), y)$ measures error.

Examples:

- 0/1 classification loss: $\ell(\hat{y}, y) = \mathbf{1}[\hat{y} \neq y]$.
- Squared regression loss: $\ell(\hat{y}, y) = (\hat{y} - y)^2$.
- Hinge loss: $\ell(yf(x)) = \max(0, 1 - yf(x))$.
- Logistic loss: $\ell(yf(x)) = \log(1 + \exp(-yf(x)))$.

Risk and empirical risk

True risk:

$$R(h) = \mathbb{E}_{(x,y) \sim P}[\ell(h(x), y)].$$

Empirical risk on sample $S = \{(x_i, y_i)\}_{i=1}^m$:

$$\hat{R}_S(h) = \frac{1}{m} \sum_{i=1}^m \ell(h(x_i), y_i).$$

Empirical risk minimization (ERM):

$$\hat{h} \in \arg \min_{h \in H} \hat{R}_S(h).$$

ERM works when empirical risk uniformly approximates true risk over H .

Hoeffding inequality

If $Z_1, \dots, Z_m$ are independent random variables in $[0, 1]$ with mean μ , then

$$\Pr \left[\left| \frac{1}{m} \sum_i Z_i - \mu \right| \geq \epsilon \right] \leq 2e^{-2m\epsilon^2}.$$

For a fixed hypothesis h , this bounds deviation between $\hat{R}_S(h)$ and $R(h)$.

For finite H , union bound gives:

$$\Pr [\exists h \in H : |\hat{R}_S(h) - R(h)| > \epsilon] \leq 2|H|e^{-2m\epsilon^2}.$$

Thus with probability at least $1 - \delta$:

$$\forall h \in H : R(h) \leq \hat{R}_S(h) + \sqrt{\frac{\ln(2|H|/\delta)}{2m}}.$$

Generalization bounds

A generalization bound upper-bounds true risk using empirical risk plus a complexity term:

$$R(h) \leq \hat{R}_S(h) + \text{complexity}(H, m, \delta).$$

For infinite classes, VC dimension replaces $\ln |H|$. If $VC(H) = d$, typical rates scale like

$$O\left(\sqrt{\frac{d + \ln(1/\delta)}{m}}\right).$$

The message: low training error is not enough; model class complexity matters.

Simple explanation

Empirical risk is training error. True risk is real-world error. Generalization bounds explain when training error is trustworthy: more data and simpler hypothesis classes make the gap between training and real-world performance smaller.

Example

If a classifier makes 3 mistakes on 100 training examples, its empirical 0/1 risk is 0.03. This does not prove the real-world error is exactly 3%; a generalization bound adds a safety margin depending on sample size and hypothesis complexity.

Statistical consistency

A learning algorithm is statistically consistent if, as $m \rightarrow \infty$, its risk converges to the best achievable risk in the considered class or to Bayes risk.

ERM is consistent under conditions such as:

- hypothesis class has finite VC dimension,
- uniform convergence holds,
- optimization solves or approximates ERM,
- the class is rich enough if consistency is meant relative to Bayes risk.

VC dimension

VC dimension is the largest number of points that can be shattered by H . It measures expressive power for binary classification.

Examples:

- Intervals on the real line: VC dimension 2.
- Linear classifiers in $\mathbb{R}^d$: VC dimension $d + 1$.

High VC dimension permits fitting many labelings and increases overfitting risk.

2. Support Vector Machines

Linear separable SVM

Binary labels $y_i \in \{-1, +1\}$, classifier

$$f(x) = w^T x + b.$$

Correct classification with margin:

$$y_i(w^T x_i + b) \geq 1.$$

Margin is $2/\|w\|$. Maximizing margin equals minimizing $\frac{1}{2}\|w\|^2$:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1.$$

Soft-margin SVM

Allow violations with slack variables $\xi_i \geq 0$:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C \sum_i \xi_i$$

subject to:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Equivalent regularized ERM with hinge loss:

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C \sum_i \max(0, 1 - y_i(w^T x_i + b)).$$

Parameter C trades margin size against training errors.

Kernel SVM

The dual form depends on dot products $x_i^T x_j$. Replace them by kernel

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

without explicitly constructing ϕ .

Common kernels:

- polynomial: $K(x, z) = (x^T z + c)^p$,
- RBF/Gaussian: $K(x, z) = \exp(-\|x - z\|^2 / (2\sigma^2))$.

Prediction:

$$f(x) = \sum_i \alpha_i y_i K(x_i, x) + b.$$

Only support vectors have $\alpha_i > 0$.

Simple explanation

An SVM is mostly determined by the borderline examples. Points far away from the separating boundary do not affect the final classifier; the support vectors are the points that hold the margin in place.

Example

In 2D, imagine two classes separated by a line. If you move a point far inside its class region, the best separating line usually does not move. If you move a point lying on the margin, the line may change; that point is a support vector.

3. Maximum Likelihood and EM

Maximum likelihood estimator

Given data $D = \{x_i\}_{i=1}^m$ and parametric model $p_\theta(x)$, likelihood:

$$L(\theta) = \prod_{i=1}^m p_\theta(x_i).$$

Log-likelihood:

$$\ell(\theta) = \sum_{i=1}^m \log p_\theta(x_i).$$

MLE:

$$\hat{\theta}_{MLE} \in \arg \max_{\theta} \ell(\theta).$$

Examples:

- Bernoulli MLE: $\hat{p} = \frac{1}{m} \sum_i x_i$.
- Gaussian mean MLE: sample mean.

Consistency of an estimator

Estimator $\hat{\theta}_m$ is consistent for θ^* if

$$\hat{\theta}_m \xrightarrow{P} \theta^*$$

as $m \rightarrow \infty$. MLE is consistent under regularity and identifiability conditions: correct model family, enough data, identifiable parameters, and suitable smoothness.

Latent-variable likelihood

For latent variables z :

$$p_\theta(x) = \sum_z p_\theta(x, z)$$

or integral for continuous z . Direct maximization of

$$\sum_i \log \sum_z p_\theta(x_i, z)$$

is often hard because of the log of a sum.

EM lower bound

For any distribution $q_i(z)$:

$$\log p_\theta(x_i) = \log \sum_z q_i(z) \frac{p_\theta(x_i, z)}{q_i(z)} \geq \sum_z q_i(z) \log \frac{p_\theta(x_i, z)}{q_i(z)}$$

by Jensen's inequality.

Define lower bound:

$$\mathcal{L}(q, \theta) = \sum_i \sum_z q_i(z) \log p_\theta(x_i, z) - \sum_i \sum_z q_i(z) \log q_i(z).$$

The gap to log-likelihood is a KL divergence:

$$\log p_\theta(D) - \mathcal{L}(q, \theta) = \sum_i KL(q_i(z) \| p_\theta(z|x_i)).$$

EM algorithm

EM performs block-coordinate ascent on the lower bound. Equivalently, if the objective is written as negative log-likelihood or negative lower bound, the same procedure can be described as block-coordinate descent.

E-step:

$$q_i(z) \leftarrow p_{\theta^{old}}(z|x_i).$$

This makes the lower bound tight at current parameters.

M-step:

$$\theta^{new} \leftarrow \arg \max_{\theta} \sum_i \mathbb{E}_{z \sim q_i} [\log p_\theta(x_i, z)].$$

EM monotonically increases likelihood, but may converge to a local optimum.

Simple explanation

EM alternates between guessing hidden variables and refitting parameters. In a Gaussian mixture, the E-step softly assigns points to clusters; the M-step recomputes cluster means, variances, and weights from those soft assignments.

Example

For a two-Gaussian mixture and one data point x , the E-step might compute responsibilities 0.8 for cluster 1 and 0.2 for cluster 2. In the M-step, x contributes 80% of its weight to updating cluster 1's mean and 20% to cluster 2's mean.

Typical applications:

- Gaussian mixture models,

- hidden Markov models,
- missing data,
- mixture of experts.

4. Deep Networks and Their Training

Neuron

A neuron computes:

$$z = w^T x + b, \quad a = \sigma(z),$$

where σ is activation function.

Common activations:

- sigmoid: $\sigma(z) = 1/(1 + e^{-z})$,
- tanh,
- ReLU: $\max(0, z)$,
- leaky ReLU,
- softmax for multiclass output.

Network architectures

Feedforward neural network:

$$h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)}).$$

Architectural types:

- MLP: fully connected layers.
- CNN: convolution, pooling, local receptive fields.
- RNN/LSTM/GRU: sequence models.
- Residual networks: skip connections to ease optimization.
- Autoencoders: learn latent representation by reconstruction.

Convolutional networks

Convolutional layer applies learned filters over spatial positions:

$$y_{k,i,j} = \sum_{c,u,v} W_{k,c,u,v} x_{c,i+u,j+v} + b_k.$$

Key ideas:

- local connectivity,
- weight sharing,
- translation equivariance,
- pooling/striding for spatial downsampling,
- many channels/features.

CNNs are standard for images and spatial signals.

Backpropagation

Backprop computes gradients by chain rule from output layer backward.

Simple explanation

Backpropagation is bookkeeping for the chain rule. First compute how wrong the output is, then move backward layer by layer, asking how much each weight contributed to that error.

For layer:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \sigma(z^{(l)}).$$

Error signal:

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}.$$

Gradients:

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)}(a^{(l-1)})^T, \quad \frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}.$$

Backward recursion:

$$\delta^{(l)} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot \sigma'(z^{(l)}).$$

Layer types

Common layers:

- dense/fully connected,
- convolution,
- pooling,
- normalization, e.g. batch normalization,
- dropout,
- recurrent layers,
- residual/skip connections,
- embedding layers.

Parameter initialization

Bad initialization can cause vanishing/exploding activations and gradients.

Common schemes:

- Xavier/Glorot for tanh/sigmoid-like activations:

$$\text{Var}(W) \approx \frac{2}{n_{in} + n_{out}}.$$

- He initialization for ReLU:

$$\text{Var}(W) \approx \frac{2}{n_{in}}.$$

Biases are often initialized to zero or small constants.

Stochastic gradient descent

Full gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta).$$

SGD estimates gradient from minibatch B :

$$\theta \leftarrow \theta - \eta \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell_i(\theta).$$

Practical improvements:

- momentum,
- Adam/RMSProp,
- learning-rate schedules,
- weight decay,
- early stopping,
- data augmentation,
- normalization.

Overfitting and generalization in deep networks

Deep networks can overfit because they have many parameters. Regularization methods:

- explicit penalties: L_2 /weight decay,
- dropout,
- data augmentation,
- early stopping,
- batch normalization effects,
- architecture constraints.

Even overparameterized networks can generalize well, but exam answers should emphasize training/validation split and regularization.

Exam Checklist

- Define risk, empirical risk, ERM.
- State Hoeffding inequality and finite-class generalization bound.
- Explain consistency and VC dimension.
- Formulate hard/soft-margin SVM and kernel trick.
- Define MLE and consistency.
- Derive EM as lower-bound maximization with E-step and M-step.
- Explain neurons, MLPs, CNNs, backpropagation, layer types.
- Explain initialization and SGD/minibatch training.